BSc (Hons) in **Information Technology Specialized in AI**

Faculty of Computing

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Practical 03

> **Objective & Lecture Mapping:** In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

> Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:

```
'grid_size': (self.width, self.height)
'walls': list(self.walls)
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

> You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent`.
2. Import required structures: `from collections import deque` and `import heapq`.
3. **BFS:** Create `def bfs_search(...)`. Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)`. Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)`. Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

> The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__`, add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'`.
2. In `sense_and_act(self, percept)`, check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']`, execute the search method matching `self.active_algo`, and store the resulting sequence of actions in `self.plan`.
4. Return the first action from the plan using `return self.plan.pop(0)`.
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

> Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

The state space is the set of all possible situations the agent can be in, together with the actions that move it from one state to another. In this lab, each state is one grid cell (x, y) that is not a wall, and the actions are Up, Down, Left and Right. It is a graph, and every cell appears in it only once. The search tree is the structure the algorithm builds while it searches. Its root is the start state, and every node stands for a path from the start, so it records a state, its parent, the action used, and the path cost. The same grid cell can appear many times in the search tree, reached through different paths. The state space is fixed and finite, but the search tree can be much larger, and without a check for repeated states it can even be infinite.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

The reached set solves the problem of repeated states. A grid has cycles, since you can go up and then down and be back where you started. Without any memory, the search treats every revisit as a brand new node, so the same cells are expanded again and again. The reached set records the cells that have already been seen, so each state is expanded at most once. This turns tree search into graph search. Without it, the DFS agent would go deeper and deeper in one direction, for example moving Right and Left back and forth, and never finish. It would loop forever, use more and more memory, and the agent would never form a plan.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

BFS expands nodes level by level, so it checks every path of length 1, then every path of length 2, and so on. In this grid every move costs the same (1), so the first time BFS reaches the food it has used the fewest possible steps. That is why BFS is optimal here, and it gives a direct, straight route. UCS gives the same route because with equal step costs it expands nodes in the same order. DFS always follows one branch as deep as it can before backing up. It stops at the first path that reaches the food, not the shortest one, and the order in which it tries directions decides which path that is. In the simulation DFS therefore took long, winding routes that crossed large parts of the grid, and on a 12x12 grid it used up the step limit without eating all the food. DFS is not optimal and only guarantees finding some path.

4. (Evaluate) If we scaled `visual_grid_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

Let b be the branching factor, d the depth of the shallowest goal, and m the maximum depth of the search tree. BFS needs time O(b^d) and space O(b^d), because it must keep the whole frontier of nodes at the current depth in memory. UCS has similar bounds in terms of the optimal cost. On a 1000x1000 grid, with up to a million cells and a frontier that grows like a wide ring around the start, this memory is the bottleneck, and the program can run out of memory before it finds food. DFS has time O(b^m) but needs only O(bm) space, because it only stores the current path and the unexpanded siblings along it. With a reached set, the space is still limited by the number of cells, but its frontier stays much smaller than BFS's. So BFS is limited by memory, while DFS is limited mainly by time and by the risk of poor, very long paths. DFS saves memory but gives up optimality and, without a reached set, even completeness.

Once Completed Get a PDF of the completed File and Push into the Week

03 brach in the Github Project

Finalize and Lock Lab 03 Documentation

FACULTY OF COMPUTING